



AGENT EVALS

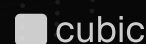
WRITE EVALS LIKE YOU WRITE FUNCTIONS.

When you change a model, prompt, or version, how do you know if it got better? For most teams, that answer requires weeks of building: defining what to capture, building pipeline handlers, and maintaining a completely separate data layer.

Inngest brings agent infrastructure into your codebase. Because Inngest captures every detail during execution, there's no separate data pipeline to build—the eval layer inherits everything the execution layer already has. Scores, live experiments, usage, p99s, and cost all surface automatically.

The result is the first eval practice that ships with your code, and scales with your agent.

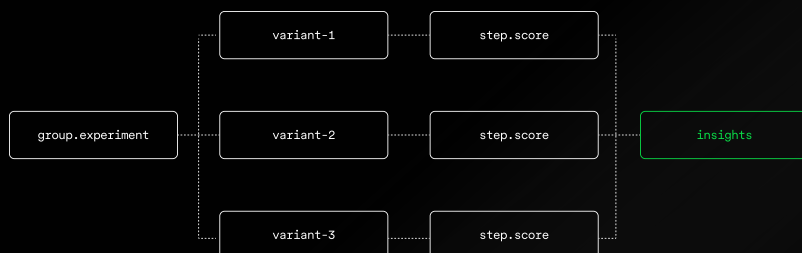
Trusted in Production by



Evals belong in your codebase

What makes Inngest Agent Evals different?

Other eval platforms score what you explicitly send them, at eval time, against whatever you remembered to capture. Inngest scores from inside execution, where every step input, output, timing, and retry already exists. And because scorers are durable functions, they can wait for real-world outcomes: Did the user convert? Did they come back? Did they give a thumbs up? Score based on what actually happened, not whether one LLM said another did well enough.



The Four Primitives

Everything you need to build agent evals in your existing codebase.

1. EXPERIMENTS

`group.experiment()`

Run two or more versions on real production traffic. No feature flag required.

2. SCORING

`step.score()`

Attach scores to any run, step, or experiment variant. Aggregate in Insights.

3. DEFER

`defer()`

Follow-up work defined and invoked from the parent function. No handler required.

4. INSIGHTS

`/insights`

Run two or more versions on real production traffic. No feature flag required.

How the whole team uses Agent Evals



AI Engineers

- Swap a model and compare output quality, latency & cost side-by-side
- Find the prompt variant that produces best results before committing
- Run old vs new providers during a migration to validate parity



Product Engineers

- Gradually roll out step variants with live output comparisons
- Use previous experiment results to justify provider or model choices
- Test a new API call with production traffic and compare p99 latency



Engineering Leaders

- Use actual business outcomes to validate model and prompt changes
- View error rates and cost across variants to improve shipping decisions
- During an incident, shift all traffic to a variant without a redeploy

Get Started

Add Evals to an existing Inngest function in three steps

1 Add an Experiment

Replace a `step.run()` with `group.experiment()`. Define your variants, and set a weighted rollout. Inngest handles deterministic variant selection per run.

2 Define a Scorer

Create a reusable scorer with `createScorer()`. Define the input schema and a handler—which can use steps, run in parallel, or wait for external signals. Call it with one line.

3 See it in Insights

Open Insights to view your experiment data—variant distributions, p99s, error rates, and scores, aggregated across every run. No export, no new instrumentation.

```
const quality = createScorer("summary-quality", {
  handler: ({ summary }) =>
    step.run("score", () => scoreWithLLM(summary)),
});

const { result, variant } = await
group.experiment("summary-model", {
  variants: {
    gpt4: () => step.run("gpt4", () =>
      summarizeWithGPT4(prompt)),
    claude: () => step.run("claude", () =>
      summarizeWithClaude(prompt)),
  },
  select: experiment.weighted({ gpt4: 70, claude:
30 }),
  withVariant: true,
});

await quality.score({ summary: result, variant });
```



INNGEST.COM
2026

